

Universidad de las Fuerzas Armadas ESPE

Sede Sangolquí

Carrera: Ingeniería de Software

NRC: 36873

Tarea 1.2: Desarrollo de una aplicación móvil con principios de diseño – Pago del servicio de agua potable

Integrantes:

Marcelo Acuña

Abner Arboleda

Christian Bonifaz

Esteban Santos

Materia: Desarrollo de Aplicaciones Móviles

Docente: Ing. Doris Chicaiza .Mgtr

Sangolquí, 23 de octubre de 2026

Índice

1. Introducción **2**

2. Objetivos **2**
2.1. Objetivo general 2
2.2. Objetivos específicos 2

3. Trabajos relacionados **3**

4. Desarrollo **3**
4.1. Conceptos básicos 3
4.2. Herramientas 4
4.2.1. Android Studio 4
4.2.2. Flutter y Dart 4
4.2.3. Visual Studio Code 4
4.3. Requisitos de la aplicación 5
4.4. Procedimiento de desarrollo 5
4.5. Principios de diseño aplicados 22

5. Conclusiones **23**

6. Recomendaciones **23**

Anexos **23**

1. Introducción

Una aplicación móvil no solo debe calcular bien: también debe ser fácil de usar, verse igual en todas sus pantallas y evitar que el usuario cometa errores al ingresar datos. Para lograrlo se aplican principios de diseño de interfaces, como la consistencia, la jerarquía visual, la retroalimentación inmediata y la prevención de errores (Nielsen, 1994; Google, s.f.-b). Además, el código se mantiene ordenado cuando se separan los datos, la lógica y la interfaz con el patrón Modelo–Vista–Controlador (MVC) (Krasner y Pope, 1988), y cuando los componentes visuales se construyen de lo simple a lo complejo con la metodología *Atomic Design* (Frost, 2016).

Este informe documenta el desarrollo de **AquaPago**, una aplicación en Flutter y Dart que simula la consulta y el cálculo del valor a pagar por el consumo de agua potable. La aplicación tiene tres pantallas: inicio de sesión, registro del consumo y detalle del pago. A partir de las lecturas anterior y actual del medidor y del tipo de consumidor (residencial o comercial), calcula el consumo en m^3, aplica la tarifa del rango correspondiente, suma el cargo fijo y el recargo y muestra el total a pagar. El proyecto se organizó con MVC, la vista con *Atomic Design* y un tema propio, y la navegación se realiza con rutas nombradas, tomando como referencia el proyecto `pry004_sueldo_rutas` trabajado en clase.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación móvil en Flutter y Dart para el cálculo del pago del servicio de agua potable, mediante el patrón Modelo–Vista–Controlador, componentes organizados con *Atomic Design*, un tema visual propio, navegación por rutas nombradas y validación de entradas, para aplicar principios de diseño en una aplicación de varias pantallas que sea fácil de usar y de mantener.

2.2. Objetivos específicos

- Organizar el proyecto en las capas modelo, vista y controlador, de modo que los cálculos y las validaciones no dependan de la interfaz.
- Construir la interfaz con átomos, moléculas, organismos y pantallas reutilizables.
- Definir un tema propio con colores, tipografía, estilos de botones y estilos de campos de entrada aplicado a toda la aplicación.
- Implementar el cálculo del consumo, la tarifa por rango, el cargo fijo, el recargo y el total a pagar.
- Validar cada dato ingresado y mostrar mensajes claros cuando un dato es incorrecto.

- Comprobar el funcionamiento de la aplicación con pruebas unitarias, pruebas de *widgets* y ejecución en un emulador Android.

3. Trabajos relacionados

1. Quisaguano et al. (2022) comparan Flutter, Xamarin y Android Studio como entornos de desarrollo móvil y describen la preparación de un proyecto Flutter en Windows con Visual Studio Code. Es el antecedente del entorno usado en este trabajo, configurado en el laboratorio 1.1.

2. Fentaw (2020) compara React Native y Flutter construyendo la misma aplicación con varias pantallas y consumo de servicios. Su análisis de la estructura de ambas aplicaciones respalda el uso de Flutter para aplicaciones con navegación entre vistas, como el flujo de inicio de sesión, registro y detalle de AquaPago.

3. George et al. (2025) contrastan Android nativo, Flutter y React Native mediante una aplicación de control de gastos, un dominio de cálculos monetarios cercano al cálculo de cargos, recargos y totales de este trabajo. Concluyen que Flutter ofrece buena productividad sin sacrificar la calidad del código.

4. Torres Peña y Andrade Solórzano (2021) desarrollaron con Flutter una aplicación informativa y de apoyo académico para la Universidad Politécnica Salesiana, organizada en módulos a los que se accede desde una pantalla principal. Su organización por módulos y pantallas es similar a la separación por capas y componentes aplicada aquí.

5. Safavi (2025) evalúa el rendimiento de React Native y Flutter y construye con Flutter un prototipo de gestión de bodega basado en formularios y listas. Sus resultados muestran que Flutter mantiene fluidas las interfaces con muchos campos y validaciones, como el formulario de registro del consumo de esta aplicación.

4. Desarrollo

4.1. Conceptos básicos

- **Modelo–Vista–Controlador (MVC):** patrón que divide una aplicación en tres partes: el *modelo* guarda los datos, la *vista* los muestra y el *controlador* procesa las acciones del usuario y aplica las reglas de negocio (Krasner y Pope, 1988).

- ***Atomic Design*:** metodología que construye la interfaz por niveles: átomos (piezas mínimas como un botón o un campo), moléculas (grupos de átomos), organismos (secciones completas), plantillas y páginas o pantallas (Frost, 2016).

- ***Widget*:** unidad básica de la interfaz en Flutter; toda pantalla, botón o texto es un *widget* (Flutter Team, s.f.-c).
- ***StatelessWidget* y *StatefulWidget*:** el primero no cambia después de construirse; el segundo guarda un estado y se vuelve a dibujar cuando se llama a `setState()` (Flutter Team, s.f.-a).
- **Tema (`ThemeData`):** objeto que define colores, tipografía y estilos de los componentes para toda la aplicación, de modo que no se repitan estilos en cada pantalla (Flutter Team, s.f.-g).
- **`Form` y `TextFormField`:** un `Form` agrupa campos de entrada; cada `TextFormField` recibe una función `validator` que devuelve un mensaje de error o `null` si el dato es correcto (Flutter Team, s.f.-b).
- **Rutas nombradas:** cadenas como `/consumo` asociadas a una pantalla. `Navigator.pushNamed` abre una pantalla y puede enviarle argumentos; `Navigator.pop` la cierra y puede devolver un valor (Flutter Team, s.f.-d,s).
- **Principios de diseño:** pautas para que una interfaz sea comprensible, como mantener la consistencia, dar retroalimentación del estado del sistema y prevenir errores antes de que ocurran (Nielsen, 1994).

4.2. Herramientas

4.2.1. Android Studio

Proporciona el SDK de Android, las herramientas de compilación y el emulador en el que se ejecutó la aplicación (Google, s.f.-a). Se usó la misma instalación configurada en el laboratorio 1.1.

4.2.2. Flutter y Dart

Se utilizó Flutter 3.47.7 (canal *stable*) con Dart 3.13.5. Flutter aporta los *widgets* de Material Design 3, el sistema de navegación, los formularios con validación y el paquete `flutter_test`; Dart es el lenguaje en el que se escribieron el modelo, los controladores y las vistas (Flutter Team, s.f.-c; Dart Team, s.f.).

4.2.3. Visual Studio Code

Editor principal del proyecto, con las extensiones de Flutter y Dart para autocompletado, análisis de código, ejecución en el emulador y *hot reload* (Microsoft, s.f.).

4.3. Requisitos de la aplicación

La Tabla 1 relaciona cada requisito de la tarea con la parte del proyecto que lo cumple.

Cuadro 1. Requisitos de la tarea y su implementación.

Requisito	**Implementación**
Pantalla de inicio de sesión	`LoginPantalla`, ruta `/`.
Pantalla principal	`ConsumoPantalla`, ruta `/consumo`: registro del consumo.
Pantalla de resultado	`DetallePagoPantalla`, ruta `/detalle`: consumo, cargos y total.
Atomic Design	Carpetas `vista/atomos`, `vista/moleculas`, `vista/organismos` y `vista/pantallas`.
Patrón MVC	Carpetas `modelo/`, `vista/` y `controlador/`.
Tema propio	`vista/tema/tema_app.dart`: colores, tipografía, botones y campos de entrada.
Navegación mediante rutas	Mapa `routes` en `main.dart`; `pushNamed`, `pushReplacementNamed` y `pop` con argumentos.
Validación de entradas	`Form` con validadores definidos en los controladores y filtros que solo aceptan dígitos.
Controles Flutter	`TextFormField`, `DropdownButtonFormField`, `ElevatedButton`, `OutlinedButton`, `IconButton`, `Card`, `ExpansionTile`, `AlertDialog`, `SnackBar`, `AppBar`, `SingleChildScrollView`.

4.4. Procedimiento de desarrollo

Paso 1: Crear el proyecto

Se creó el proyecto desde la terminal, dentro de la carpeta de la unidad:

```
flutter create tarea1_2
cd tarea1_2
```

Todo el código propio se escribió dentro de `lib/`, y el archivo `lib/main.dart` generado por Flutter se reemplazó por el punto de entrada de la aplicación.

Paso 2: Organizar el proyecto en MVC y Atomic Design

Siguiendo la estructura del proyecto `pry004_sueldo_rutas`, el código se dividió en las tres capas de MVC, y la capa de vista se dividió a su vez en los niveles de *Atomic Design*.

Código 1. Estructura de carpetas del proyecto.

```
tarea1_2/
```

```
|-- lib/
|   |-- main.dart                    Punto de entrada: tema y rutas
|   |-- modelo/                      Datos de la aplicación
|   |   |-- usuario_modelo.dart
|   |   |-- tipo_consumidor.dart         Residencial y comercial
|   |   |-- tarifa_modelo.dart           Rango de consumo y precio
   por m3
|   |   |-- consumo_modelo.dart          Datos ingresados
|   |   `-- pago_modelo.dart             Resultado del cálculo
|   |-- controlador/                 Reglas de negocio y validaciones
|   |   |-- login_controlador.dart
|   |   `-- pago_controlador.dart
|   `-- vista/
|       |-- tema/tema_app.dart       Colores, tipografía, botones y
   campos
|       |-- utilidades/formato.dart  Formato de moneda y porcentaje
|       |-- atomos/                  Botones, campos, selector,
   textos, logo
|       |-- moleculas/               Grupos de campos, filas de
   detalle, total
|       |-- organismos/              Formularios y tarjetas
|       `-- pantallas/               Login, consumo y detalle de pago
|-- test/
|   `-- widget_test.dart             Pruebas unitarias y de widgets
`-- pubspec.yaml
```

Las vistas solo muestran datos y avisan de las acciones del usuario; los cálculos y las validaciones están en los controladores, y los datos viajan entre capas como objetos del modelo.



Figura 1. Estructura del proyecto en Visual Studio Code.

Paso 3: Definir el modelo

El modelo contiene las clases que representan los datos: el usuario, el tipo de consumidor, la tarifa de cada rango, los datos del consumo ingresados por el usuario y el resultado del pago. El tipo de consumidor se definió como un `enum` que guarda su cargo fijo y su porcentaje de recargo:

Código 2. Tipos de consumidor (`tipo_consumidor.dart`).

```dart
// Tipos de consumidor con su cargo fijo y porcentaje de recargo
enum TipoConsumidor {
  residencial(nombre: 'Residencial', cargoFijo: 3.00,
   porcentajeRecargo: 0.0),
  comercial(nombre: 'Comercial', cargoFijo: 6.00, porcentajeRecargo:
   0.10);

  final String nombre;
  final double cargoFijo;
  final double porcentajeRecargo;

  const TipoConsumidor({
    required this.nombre,
    required this.cargoFijo,
    required this.porcentajeRecargo,
```

```
  });
}
```

Cada rango de consumo es un `TarifaModelo` que sabe si un consumo pertenece a él; un límite superior nulo indica que el rango no tiene tope:

Código 3. Rango de consumo y su tarifa (`tarifa_modelo.dart`).

```dart
class TarifaModelo {
  final String nombre;
  final int desde;
  final int? hasta; // null = sin límite superior
  final double precioM3;

  const TarifaModelo({required this.nombre, required this.desde, this
    .hasta, required this.precioM3});

  bool incluye(int consumo) =>
      consumo >= desde && (hasta == null || consumo <= hasta!);

  String get rango =>
      hasta == null ? 'más de ${desde - 1} m³' : '$desde a $hasta m³'
    ;
}
```

Las tarifas no corresponden a una empresa pública real; se definieron para la práctica y se resumen en la Tabla 2.

Cuadro 2. Valores definidos para la práctica.

Rango de consumo	Tarifa por m³	Tipo	Cargo fijo	Recargo
0 a 15 m³ (básica)	$0.45	Residencial	$3.00	0 %
16 a 30 m³ (intermedia)	$0.70	Comercial	$6.00	10 %
Más de 30 m³ (superior)	$1.05			

Paso 4: Implementar los controladores

Controlador de inicio de sesión. `LoginControlador` valida que el usuario no esté vacío, no tenga espacios y tenga al menos 4 caracteres, que la contraseña tenga al menos 4 caracteres, y compara las credenciales con un usuario de prueba (`admin` / `1234`).

Controlador de pago. `PagoControlador` contiene la lista de tarifas, las validaciones del formulario de consumo y el cálculo del pago. El cálculo sigue las fórmulas del enunciado: Consumo = Lectura actual - Lectura anterior
Valor consumo = Consumo $\times tarifa\ por\ m^3$

$Subtotal = Valor\ consumo + cargo\ fijo$
$Recargo = Subtotal \times porcentaje\ de\ recargo$
$Total\ a\ pagar = Subtotal + Recargo$

La tarifa se elige según el rango en el que cae el consumo y se aplica a todos los m^3 consumidos. Cada valor monetario se redondea a dos decimales para evitar errores de precisión de los números de punto flotante.

Código 4. Tarifas y cálculo del pago (fragmento de `pago_controlador.dart`).

```
class PagoControlador {
  // Rangos de consumo definidos para la práctica
  static const List<TarifaModelo> tarifas = [
    TarifaModelo(nombre: 'Básica', desde: 0, hasta: 15, precioM3:
   0.45),
    TarifaModelo(nombre: 'Intermedia', desde: 16, hasta: 30, precioM3
   : 0.70),
    TarifaModelo(nombre: 'Superior', desde: 31, precioM3: 1.05),
  ];

  //metodo obtener tarifa segun el rango de consumo
  TarifaModelo obtenerTarifa(int consumo) {
    return tarifas.firstWhere((tarifa) => tarifa.incluye(consumo));
  }

  //metodo calcular pago
  PagoModelo calcularPago(ConsumoModelo datos) {
    int consumo = datos.lecturaActual - datos.lecturaAnterior;
    TarifaModelo tarifa = obtenerTarifa(consumo);

    double valorConsumo = _redondear(consumo * tarifa.precioM3);
    double cargoFijo = datos.tipo.cargoFijo;
    double subtotal = _redondear(valorConsumo + cargoFijo);
    double recargo = _redondear(subtotal * datos.tipo.
   porcentajeRecargo);
    double total = _redondear(subtotal + recargo);

    return PagoModelo(
      datos: datos,
      consumo: consumo,
      tarifa: tarifa,
      valorConsumo: valorConsumo,
      cargoFijo: cargoFijo,
      subtotal: subtotal,
```

```
      porcentajeRecargo: datos.tipo.porcentajeRecargo,
      recargo: recargo,
      total: total,
    );
  }

  double _redondear(double valor) => (valor * 100).round() / 100;
}
```

La Figura 2 representa el algoritmo del cálculo.

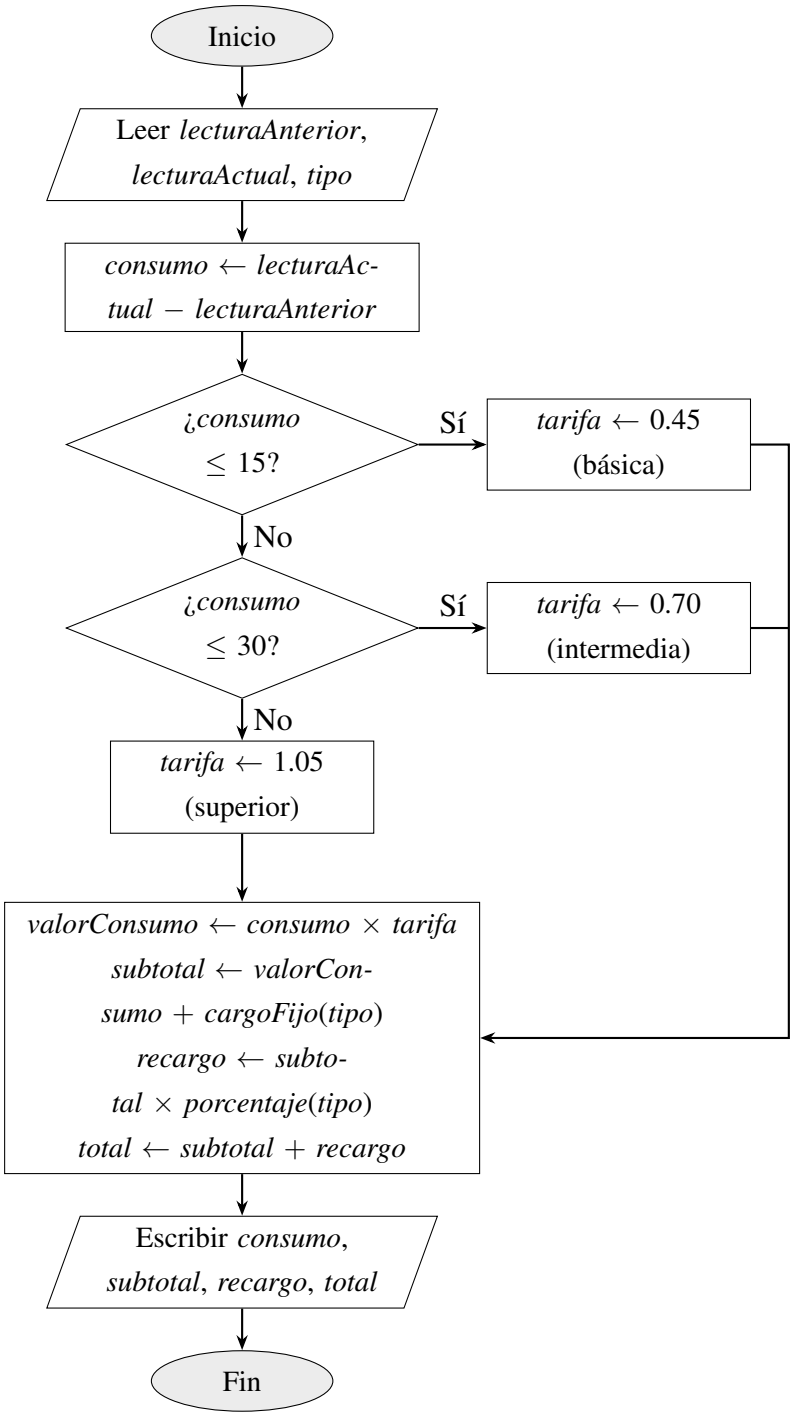


Figura 2. Diagrama de flujo del cálculo del valor a pagar.

Validaciones. Cada validador devuelve un mensaje de error o `null` si el dato es correcto, y la vista solo lo asigna al campo. La validación de la lectura actual recibe también el texto de la lectura anterior para compararlas:

Código 5. Validación de la lectura actual (fragmento de `pago_controlador.dart`).

```
String? validarLecturaActual(String? valor, String lecturaAnterior) {
  final texto = valor?.trim() ?? '';
  if (texto.isEmpty) return 'Ingrese la lectura actual';
```

```
  final actual = int.tryParse(texto);
  if (actual == null) return 'Ingrese un número entero';
  final anterior = int.tryParse(lecturaAnterior.trim());
  if (anterior != null && actual < anterior) {
    return 'Debe ser mayor o igual a la anterior';
  }
  return null;
}
```

La Tabla 3 resume todas las reglas de validación.

Cuadro 3. Reglas de validación de las entradas.

Campo	**Regla**	**Mensaje**
Usuario	Obligatorio, sin espacios, mínimo 4 caracteres.	“Ingrese su usuario”
Contraseña	Obligatoria, mínimo 4 caracteres.	“Ingrese su contraseña”
Credenciales	Deben coincidir con el usuario de prueba.	“Usuario o contraseña incorrectos”
Número de cuenta	Solo dígitos, entre 6 y 10.	“La cuenta debe tener entre 6 y 10 dígitos”
Nombre del cliente	Solo letras (con tildes) y espacios, mínimo 3 caracteres.	“Solo se permiten letras y espacios”
Tipo de consumidor	Debe seleccionarse una opción.	“Seleccione el tipo de consumidor”
Lectura anterior	Obligatoria, número entero.	“Ingrese la lectura anterior”
Lectura actual	Obligatoria, entero, mayor o igual a la anterior.	“Debe ser mayor o igual a la anterior”

Paso 5: Crear el tema propio

En `vista/tema/tema_app.dart` se definió la clase `AppColores` con una paleta de tonos azules asociados al agua, y la clase `TemaApp`, que devuelve el `ThemeData` de la aplicación con Material 3 (Flutter Team, s.f.-g; Google, s.f.-b). La Tabla 4 muestra los colores y la Tabla 5 la tipografía.

Cuadro 4. Paleta de colores del tema.

Color	Código	Uso
`primario`	`#0277BD`	Barra superior, botones, íconos y bordes de foco.
`primarioOscuro`	`#01579B`	Título de la aplicación y degradados.
`secundario`	`#00ACC1`	Degradado del logo y del total a pagar.
`fondo`	`#F1F8FC`	Fondo de las pantallas.
`texto`	`#0D2A3F`	Texto principal.
`textoSuave`	`#5B7083`	Texto secundario y etiquetas.
`borde`	`#B3D4E8`	Bordes de campos y divisores.
`error`	`#D32F2F`	Bordes y mensajes de error.
`exito`	`#2E7D32`	Ícono de pago registrado.

Cuadro 5. Tipografía del tema.

Estilo	Tamaño y peso	Uso
`headlineMedium`	30 pt, negrita	Nombre de la aplicación en el inicio de sesión.
`titleLarge`	20 pt, w700	Bienvenida y total en el detalle.
`titleMedium`	16 pt, w700, color primario	Títulos de sección.
`bodyLarge`	16 pt	Valores de los resultados.
`bodyMedium`	14 pt, color suave	Textos de apoyo y etiquetas.
`labelLarge`	16 pt, w600	Texto de los botones.

Los botones principales son rellenos y ocupan todo el ancho; los secundarios tienen solo borde, para distinguir la acción principal de la secundaria. Los campos de entrada tienen fondo blanco, bordes redondeados, ícono del color primario y cambian de color al recibir el foco o al tener un error:

Código 6. Estilos de botones y campos de entrada (fragmento de `tema_app.dart`).

```
// Estilos de botones
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: AppColores.primario,
    foregroundColor: Colors.white,
    minimumSize: const Size.fromHeight(52),
    elevation: 2,
    shape: formaBoton,
    textStyle: textoBoton,
  ),
),
outlinedButtonTheme: OutlinedButtonThemeData(
```

```
  style: OutlinedButton.styleFrom(
    foregroundColor: AppColores.primario,
    minimumSize: const Size.fromHeight(52),
    side: const BorderSide(color: AppColores.primario, width: 1.5),
    shape: formaBoton,
    textStyle: textoBoton,
  ),
),

// Estilos de campos de entrada
inputDecorationTheme: InputDecorationTheme(
  filled: true,
  fillColor: AppColores.superficie,
  prefixIconColor: AppColores.primario,
  errorMaxLines: 2,
  border: bordeBase,
  enabledBorder: bordeBase,
  focusedBorder: bordeBase.copyWith(
    borderSide: const BorderSide(color: AppColores.primario, width:
   2),
  ),
  errorBorder: bordeBase.copyWith(
    borderSide: const BorderSide(color: AppColores.error),
  ),
),
```

Paso 6: Construir los componentes con Atomic Design

La vista se construyó de lo simple a lo complejo. Los átomos reciben sus datos y validadores por parámetro y no conocen la pantalla en la que se usan, por lo que se reutilizan en varias pantallas. La Tabla 6 resume los componentes de cada nivel.

Cuadro 6. Componentes de la interfaz por nivel de *Atomic Design*.

Nivel	Componentes	Uso
Átomos	`BotonPrincipal`, `BotonSecundario`, `CampoTexto`, `CampoNumero`, `CampoClave`, `SelectorTipo`, `TextoTitulo`, `TextoSeccion`, `IconoLogo`	Botones, campos de texto, numéricos y de contraseña, lista desplegable, títulos y logo.
Moléculas	`EncabezadoApp`, `EntradaCredenciales`, `EntradaCliente`, `EntradaLecturas`, `FilaDetalle`, `ResumenTotal`	Logo con título, grupos de campos, filas etiqueta–valor y total destacado.
Organismos	`FormularioLogin`, `FormularioConsumo`, `TarjetaTarifas`, `TarjetaDetallePago`	Formularios completos, tabla de tarifas desplegable y desglose del pago.
Pantallas	`LoginPantalla`, `ConsumoPantalla`, `DetallePagoPantalla`	Pantallas completas a las que apuntan las rutas.

Por ejemplo, el átomo `CampoNumero` solo acepta dígitos, limita la longitud y valida mientras el usuario escribe; se usa para el número de cuenta y para las dos lecturas:

Código 7. Átomo `CampoNumero` (`campo_numero.dart`).

```
class CampoNumero extends StatelessWidget {
  final TextEditingController controller;
  final String etiqueta;
  final IconData icono;
  final FormFieldValidator<String>? validator;
  final String? sufijo;
  final int longitudMaxima;

  const CampoNumero({super.key, required this.controller, required
   this.etiqueta,
      required this.icono, this.validator, this.sufijo, this.
   longitudMaxima = 7});

  @override
  Widget build(BuildContext context) {
    return TextFormField(
      controller: controller,
```

```
      keyboardType: TextInputType.number,
      inputFormatters: [FilteringTextInputFormatter.digitsOnly],
      maxLength: longitudMaxima,
      textInputAction: TextInputAction.next,
      autovalidateMode: AutovalidateMode.onUserInteraction,
      validator: validator,
      decoration: InputDecoration(
        labelText: etiqueta,
        prefixIcon: Icon(icono),
        suffixText: sufijo,
        counterText: '',
      ),
    );
  }
}
```

La molécula `EntradaLecturas` combina dos `CampoNumero` en una fila, y el organismo `FormularioConsumo` agrupa en un `Form` las moléculas `EntradaCliente` y `EntradaLecturas`, el átomo `SelectorTipo` y los botones *Calcular pago* y *Limpiar*.

Paso 7: Configurar la navegación mediante rutas

En `main.dart` se registraron las tres rutas nombradas y se aplicó el tema propio:

Código 8. Punto de entrada y rutas de la aplicación (`main.dart`).

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'AquaPago',
      theme: TemaApp.tema,
      initialRoute: '/',
      routes: {
        '/': (context) => const LoginPantalla(),
        '/consumo': (context) => const ConsumoPantalla(),
        '/detalle': (context) => const DetallePagoPantalla(),
      },
    );
  }
```

```
}
```

La Figura 3 muestra cómo se pasa de una pantalla a otra. Tras iniciar sesión se usa `pushReplacementNamed`, para que el botón atrás no regrese al inicio de sesión. Al calcular, la pantalla de consumo envía el objeto `PagoModelo` completo como argumento a la ruta `/detalle` y espera su respuesta: si el pago se confirma, la pantalla de detalle devuelve `true` y el formulario se limpia.

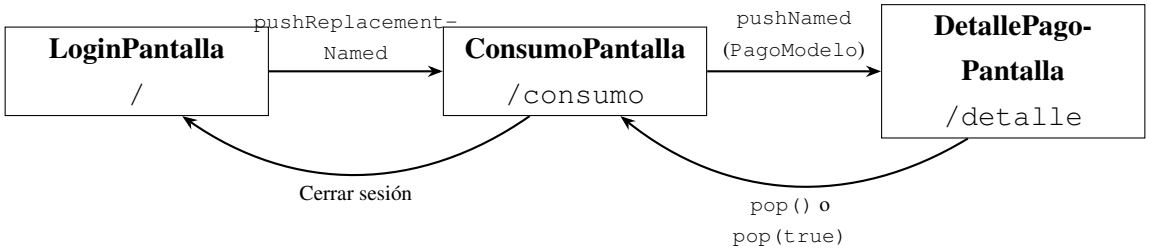


Figura 3. Navegación entre pantallas mediante rutas nombradas.

Código 9. Envío de datos a la pantalla de detalle (fragmento de `consumo_pantalla.dart`).

```dart
Future<void> calcularPago() async {
  if (!formKey.currentState!.validate()) return;

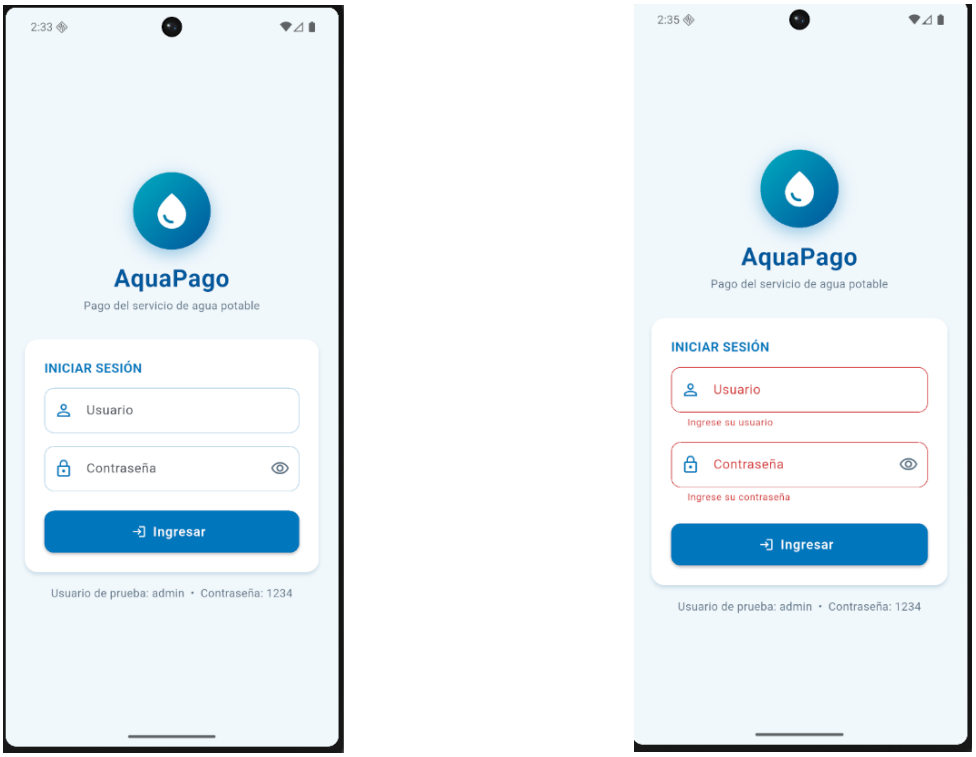
  final datos = ConsumoModelo(
    numeroCuenta: cuentaController.text.trim(),
    nombreCliente: nombreController.text.trim(),
    lecturaAnterior: int.parse(anteriorController.text),
    lecturaActual: int.parse(actualController.text),
    tipo: tipo!,
  );
  final resultado = controlador.calcularPago(datos);

  //paso de parametros, la pantalla de detalle devuelve true si se
   pagó
  final pagado = await Navigator.pushNamed(
    context,
    '/detalle',
    arguments: resultado,
  );
  if (!mounted) return;
  if (pagado == true) limpiar();
}
```

En la pantalla de detalle, el objeto se recupera con `ModalRoute.of(context)!.settings.ar` `as PagoModelo`.

Paso 8: Pantalla de inicio de sesión

Muestra el logo, el nombre de la aplicación y una tarjeta con los campos de usuario y contraseña. El campo de contraseña tiene un botón para mostrar u ocultar el texto, y al pulsar *Ingresar* o la tecla de envío del teclado se validan los campos. Si las credenciales no coinciden, aparece un `SnackBar` con el mensaje de error.



(a) Pantalla inicial (b) Validación de campos vacíos

Figura 4. Pantalla de inicio de sesión.

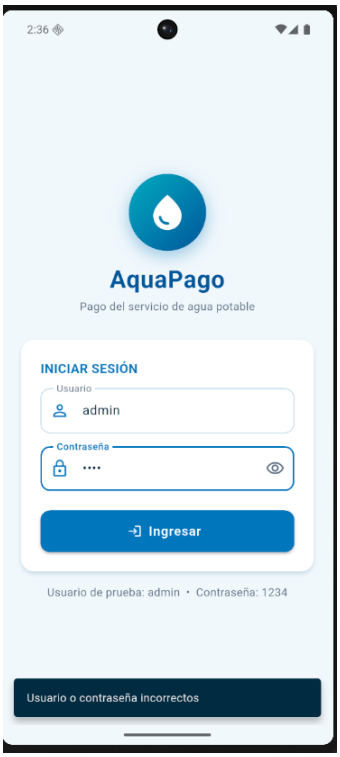
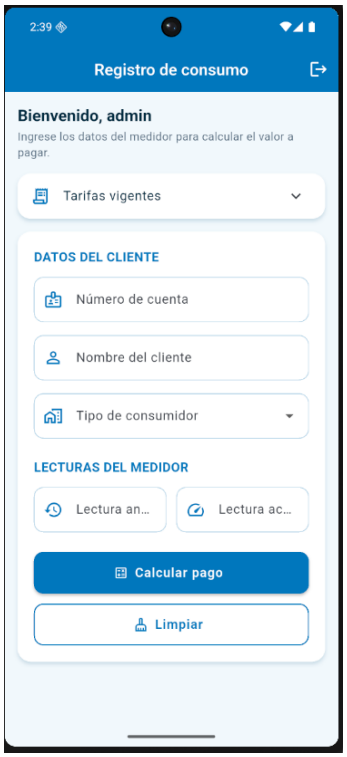


Figura 5. Mensaje de credenciales incorrectas.

Paso 9: Pantalla de registro del consumo

Es la pantalla principal. Saluda al usuario que inició sesión, ofrece una tarjeta desplegable con las tarifas vigentes y presenta el formulario con el número de cuenta, el nombre del cliente, el tipo de consumidor y las lecturas anterior y actual. El botón *Limpiar* reinicia el formulario y el ícono de la barra superior cierra la sesión.



(a) Formulario vacío



(b) Tabla de tarifas desplegada

Figura 6. Pantalla de registro del consumo.



(a) Mensajes de validación



(b) Formulario con datos válidos

Figura 7. Validación de entradas en el registro del consumo.

Paso 10: Pantalla de detalle de pago

Presenta en la parte superior el total a pagar en un recuadro con degradado y debajo una tarjeta con el desglose: datos del cliente, lecturas, consumo, rango y tarifa aplicados, valor del consumo, cargo fijo, subtotal, recargo y total. El botón *Confirmar pago* abre un `AlertDialog` que simula el registro del pago, y *Corregir datos* regresa al formulario sin borrar los datos.



(a) Consumidor comercial



(b) Consumidor residencial

Figura 8. Pantalla de detalle de pago.

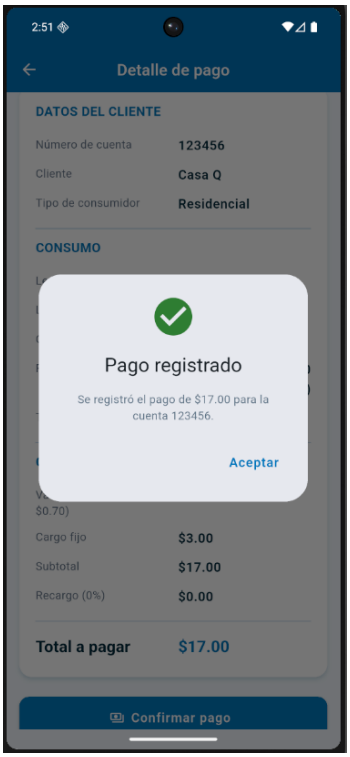


Figura 9. Confirmación simulada del pago.

4.5. Principios de diseño aplicados

- **Consistencia:** todos los colores, tipografías, botones y campos provienen del tema propio, y los mismos átomos se reutilizan en las tres pantallas (Nielsen, 1994).
- **Jerarquía visual:** el total a pagar se destaca en un recuadro grande con degradado; los títulos de sección usan el color primario y mayúsculas; el botón principal es relleno y el secundario solo tiene borde.
- **Retroalimentación:** los campos se validan mientras el usuario escribe, las credenciales incorrectas se avisan con un `SnackBar` y el pago confirmado con un diálogo.
- **Prevención de errores:** los campos numéricos solo aceptan dígitos y tienen longitud máxima, el tipo de consumidor se elige de una lista y la lectura actual no puede ser menor que la anterior.
- **Adaptación a la pantalla:** el contenido se desplaza con `SingleChildScrollView`, respeta las zonas seguras con `SafeArea` y limita su ancho con `ConstrainedBox` en pantallas grandes.

5. Conclusiones

1. Separar la aplicación en modelo, vista y controlador concentró las fórmulas, las tarifas y las validaciones en los controladores. Así se pudieron probar los cálculos sin interfaz, y cambiar una tarifa solo requiere modificar una línea del controlador sin tocar las pantallas.

2. Construir la vista con *Atomic Design* permitió reutilizar los mismos átomos (botones, campos numéricos, filas de detalle) en las tres pantallas, de modo que cada pantalla se redujo a combinar organismos ya construidos y la interfaz quedó uniforme.

3. Definir un tema propio y aplicar principios de diseño como la retroalimentación inmediata y la prevención de errores hizo que el usuario sepa en todo momento qué dato está mal y por qué, y que el total a pagar sea lo primero que se ve en el resultado.

6. Recomendaciones

1. Mantener las validaciones en el controlador y no en los *widgets*, para poder reutilizarlas en otras pantallas y probarlas con pruebas unitarias.

2. Definir los colores y estilos en un tema central desde el inicio del proyecto en lugar de escribirlos en cada *widget*, para que un cambio de diseño se aplique a toda la aplicación desde un solo archivo.

3. Pasar objetos del modelo como argumentos de las rutas en lugar de mapas con valores sueltos, y liberar los `TextEditingController` en `dispose()` para evitar fugas de memoria.

Anexos

Anexo A: Repositorio del proyecto

El código fuente completo de la aplicación está publicado en GitHub:

https://github.com/adarboleda/clases_moviles/tree/main/tarea1_2

Referencias

Dart Team. (s.f.). *Introduction to Dart*. Google. https://dart.dev/language

Fentaw, A. E. (2020). *Cross platform mobile application development: A comparison study of React Native vs Flutter* [Tesis de maestría, University of Jyväskylä]. JYX. https://urn.fi/URN:NBN:fi:jyu-202006295155

Flutter Team. (s.f.-a). *Adding interactivity to your Flutter app*. Google. https://docs.flutter.dev/ui/interactivity

Flutter Team. (s.f.-b). *Build a form with validation*. Google. https://docs.flutter.dev/cookbook/forms/validation

Flutter Team. (s.f.-c). *Flutter documentation*. Google. https://docs.flutter.dev

Flutter Team. (s.f.-d). *Navigation and routing*. Google. https://docs.flutter.dev/ui/navigation

Flutter Team. (s.f.-e). *Pass arguments to a named route*. Google. https://docs.flutter.dev/cookbook/navigation/navigate-with-arguments

Flutter Team. (s.f.-f). *Testing Flutter apps*. Google. https://docs.flutter.dev/testing/overview

Flutter Team. (s.f.-g). *Use themes to share colors and font styles*. Google. https://docs.flutter.dev/cookbook/design/themes

Frost, B. (2016). *Atomic design*. Brad Frost. https://atomicdesign.bradfrost.com

George, K., Kirubanand, V. B., y Sandeep, J. (2025). Cross-platform vs. native mobile development: A comprehensive study using an expense tracker application. En *Proceedings of the 2025 International Conference on Computational Innovations and Sustainable Technologies (ICCIST 2025)*. IEEE.

Google. (s.f.-a). *Android Studio*. Android Developers. https://developer.android.com/studio

Google. (s.f.-b). *Material Design 3*. https://m3.material.io

Krasner, G. E., y Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming, 1*(3), 26–49.

Microsoft. (s.f.). *Visual Studio Code*. https://code.visualstudio.com

Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Quisaguano, L. R., Camalle, T. G., y Toca, J. W. (2022). Análisis comparativo de entornos de desarrollo móvil. *Ciencia Latina Revista Científica Multidisciplinar, 6*(4), 4478–4498. https://doi.org/10.37811/cl_rcm.v6i4.2950

Safavi, S. (2025). *Performance evaluation of cross-platform mobile frameworks: A comparative study of React Native and Flutter* [Tesis, Università degli Studi di Padova]. Padua Thesis

Archive. https://hdl.handle.net/20.500.12608/87277

Torres Peña, P. A., y Andrade Solórzano, B. A. (2021). *Aplicación móvil informativa y de apoyo académico de la Universidad Politécnica Salesiana* [Trabajo de titulación de grado, Universidad Politécnica Salesiana]. Repositorio Institucional UPS. https://dspace.ups.edu.ec/handle/123456789/21490